

Système intelligent de détection d'incendie basé sur l'IoT et l'intelligence artificielle

Projet de fin de module & recherche

Master IASD, S2 - Module : Plateformes IoT Core : Technologies, Data et IA

Université Abdelmalek Essaâdi - Faculté des Sciences et Techniques de Tanger

Réalisé par : Khaloufi Youssef, Oussama Allouch, Bourti Ayoub, ELKAIRAH Hassan, LOUAHABI Abdenour

Encadrant : Mohamed EL BRAK

Année universitaire : 2025/2026

Résumé

Ce travail présente la conception et la validation d'un prototype IoT intelligent de détection d'incendie. Le système repose sur une couche edge simulée en Python qui génère des mesures de température, fumée, gaz et humidité selon trois scénarios : normal, suspect et danger. Les données sont publiées au format JSON via MQTT vers un broker Mosquitto local, puis orchestrées dans Node-RED. Une brique d'intelligence artificielle, basée sur un Random Forest Classifier, classe l'état détecté et enrichit le message avec un statut IA, un score de risque et une alerte. Les résultats sont ensuite visualisés sur un tableau de bord ThingsBoard Cloud. Les tests réalisés montrent le fonctionnement bout en bout de la chaîne, depuis la génération des données jusqu'à la visualisation cloud et l'alerte en cas de danger.

Mots-clés : IoT, MQTT, Node-RED, Python, ThingsBoard, Intelligence Artificielle, Random Forest, détection d'incendie, JSON.

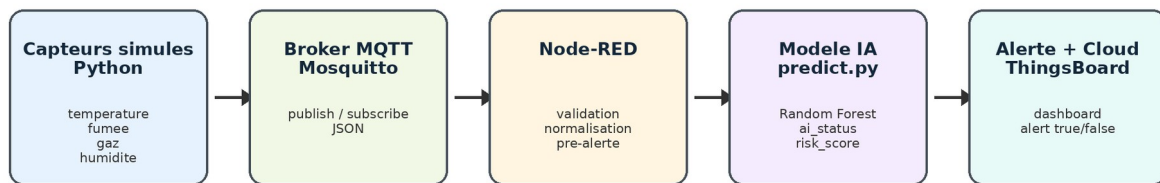
1. Introduction et contexte du projet

La surveillance des risques d'incendie constitue un cas d'usage pertinent pour illustrer l'intégration entre objets connectés, traitement de données en temps réel et intelligence artificielle. Dans un environnement réel, un tel système pourrait s'appuyer sur des capteurs de température, de fumée, de gaz et d'humidité afin d'identifier rapidement des situations anormales. Dans ce projet, l'objectif est de réaliser une démonstration académique entièrement fonctionnelle, mais sans matériel physique : les capteurs sont simulés et la chaîne de traitement est construite avec des technologies IoT standards.

Le cahier des charges du module impose la conception d'un système IoT complet de bout en bout, intégrant une couche objets/edge, une communication MQTT en mode publish/subscribe, des messages JSON, un traitement avec Node-RED, une plateforme cloud IoT et une composante d'intelligence artificielle. Le projet répond à ces exigences à travers un prototype hybride : l'acquisition et l'orchestration sont exécutées localement, tandis que la visualisation finale est documentée sur ThingsBoard Cloud.

Le choix du sujet s'inscrit dans une problématique simple et compréhensible : détecter automatiquement si l'état d'un environnement est normal, suspect ou dangereux à partir de mesures simulées. Cette approche permet de valider les concepts du module sans dépendre de capteurs physiques, tout en conservant une architecture proche d'un système IoT réel.

Architecture globale du système IoT et IA



Flux valide : donnees capteurs -> MQTT -> orchestration -> classification IA -> visualisation et alerte

Figure 1 - Architecture globale proposée pour la chaîne IoT et IA.

2. État de l'art : IoT, orchestration et IA

L'Internet des Objets repose sur l'interconnexion de dispositifs capables de produire, transmettre et exploiter des données. Dans les architectures IoT, la couche edge regroupe les capteurs, actionneurs ou passerelles proches du terrain. Ces éléments envoient généralement des données vers des services de traitement ou de visualisation, localement ou dans le cloud.

MQTT est largement utilisé dans les scénarios IoT car il s'appuie sur un modèle publish/subscribe léger. Les producteurs publient des messages sur des topics, tandis que les consommateurs s'abonnent aux topics qui les intéressent. Cette séparation entre producteurs et consommateurs simplifie la communication entre les composants d'un système distribué. Dans notre cas, le simulateur publie les données capteurs sur le topic ``iot/fire/sensor/data``, et Node-RED les reçoit en tant qu'abonné.

Node-RED est une plateforme visuelle de programmation orientée flux. Elle permet de relier des nœuds de réception, transformation, décision et sortie afin de construire rapidement des pipelines IoT. Elle est particulièrement adaptée à ce projet, car elle facilite l'orchestration entre MQTT, scripts Python et logique d'alerte.

La partie IA constitue la contribution de recherche du projet. Pour des données tabulaires simples, les modèles d'arbres et les forêts aléatoires sont des choix adaptés, car ils sont robustes, faciles à entraîner et relativement interprétables. Le modèle choisi, Random Forest Classifier, permet de classer les états normal, suspect et danger à partir de quatre variables numériques : température, fumée, gaz et humidité.

Enfin, une plateforme cloud IoT permet de stocker, visualiser ou exploiter les données. Dans ce projet, ThingsBoard Cloud est utilisé comme support de tableau de bord pour présenter les courbes de capteurs et l'état IA final.

3. Cahier des charges et objectifs fonctionnels

Le projet est un projet simulé avec intelligence artificielle. Il vise à construire une chaîne complète allant de la génération des données jusqu'à l'affichage cloud. Le système doit simuler des données de capteurs d'incendie, transmettre ces données via MQTT au format JSON, les traiter avec Node-RED, les analyser avec une brique IA, déclencher une alerte en cas de danger et afficher les informations sur un tableau de bord.

Les objectifs fonctionnels retenus sont les suivants :

- générer automatiquement des données réalistes de capteurs incendie ;
- publier les données via MQTT sur un broker local Mosquitto ;
- utiliser un format JSON commun entre tous les composants ;
- orchestrer la réception, la validation et l'enrichissement des données dans Node-RED ;
- classifier l'état du système avec un modèle IA en trois classes : normal, suspect, danger ;
- déclencher une alerte lorsque le modèle IA détecte un danger ;
- visualiser les résultats dans un tableau de bord cloud ThingsBoard.

Organisation technique du travail

Membre	Responsabilité technique	Livrables principaux
Khaloufi Youssef	Simulation edge et publication MQTT	simulate_fire_sensors.py, config.json, format JSON
Oussama Allouch	Flow Node-RED de base	réception MQTT, validation, normalisation, pre_alert
Bourti Ayoub	Dataset IA, entraînement et prédiction	dataset, train_model.py, predict.py, modèle joblib
ELKAIRAH Hassan	Intégration IA dans Node-RED	exec node, parsing JSON, fusion IA, alerte MQTT
LOUAHABI Abdenour	Cloud, dashboard et preuve de bout en bout	ThingsBoard, captures, validation cloud

4. Description de l'architecture globale

L'architecture réalisée suit un pipeline séquentiel. Le simulateur Python représente les capteurs. Il génère des messages JSON et les publie sur MQTT. Node-RED reçoit ces messages, vérifie la présence des champs obligatoires, normalise les valeurs numériques, ajoute une première estimation de risque par règles, puis appelle le script IA. La sortie IA est fusionnée avec le message original. Si le statut final est `danger`, une alerte est publiée sur le topic `iot/fire/alert`. Les données enrichies sont ensuite exploitées pour la visualisation cloud.

Cette architecture sépare clairement les responsabilités : acquisition, transport, orchestration, classification, alerte et visualisation. Ce découpage rend le prototype plus simple à tester et à présenter. Il permet aussi de remplacer plus tard le simulateur par de vrais capteurs sans modifier toute la chaîne.

5. Partie IoT : capteurs simulés, MQTT, Node-RED et cloud

5.1 Couche edge : simulateur Python

La couche edge est représentée par un script Python. Celui-ci simule quatre mesures : température, fumée, gaz et humidité. Les valeurs sont générées selon trois scénarios. Le scénario normal correspond à des valeurs faibles ou ordinaires ; le scénario suspect représente une situation inhabituelle ; le scénario danger simule un risque élevé d'incendie. Chaque message contient également un identifiant de capteur, un horodatage, une localisation et un label de scénario utilisé pour la validation.

Le simulateur lit sa configuration dans `simulator/config.json`. Le broker par défaut est `localhost:1883`, le topic capteurs est `iot/fire/sensor/data` et l'intervalle de publication est de trois secondes. Le script prévoit aussi un mode hors ligne afin de continuer à générer des JSON si MQTT n'est pas disponible.

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ sudo service mosquitto start
[sudo: authenticate] Password:
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ python simulator/simulate_fire_sensors.py
Démarrage du simulateur IoT - Membre 1
Broker MQTT : localhost:1883
Topic MQTT : iot/fire/sensor/data
Intervalle : 3 secondes

-----
Connexion au broker MQTT : localhost:1883
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:46", "temperature": 27.21, "smoke": 38, "gas": 66, "humidity": 46, "location": "Salle_1", "scenario": "normal"}
-----
Connexion MQTT réussie.
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:49", "temperature": 20.9, "smoke": 171, "gas": 205, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
-----
Message publié :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:03:52", "temperature": 23.09, "smoke": 70, "gas": 106, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
-----
Message publié :
```

Figure 2 - Publication MQTT des données générées par le simulateur Python.

5.2 Communication MQTT et format JSON

La communication est réalisée selon le modèle publish/subscribe. Le simulateur agit comme publisher et envoie les données sur un topic MQTT. Node-RED et les outils de test peuvent s'abonner à ce topic afin de recevoir les messages en temps réel. Le format JSON facilite l'interopérabilité, car chaque composant peut lire les champs par leur nom.

Un exemple de message transmis est : `device\_id`, `timestamp`, `temperature`, `smoke`, `gas`, `humidity`, `location`, `scenario`. Les champs `pre\_alert`, `ai\_status`, `risk\_score` et `alert` ne viennent pas du simulateur ; ils sont ajoutés après traitement dans Node-RED et par le modèle IA.

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ mosquitto_sub -h localhost -t iot/fire/sensor/data
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:53", "temperature": 31.07, "smoke": 77, "gas": 71, "humidity": 62, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:56", "temperature": 25.47, "smoke": 138, "gas": 154, "humidity": 54, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:04:59", "temperature": 84.95, "smoke": 906, "gas": 673, "humidity": 29, "location": "Salle_1", "scenario": "danger"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:02", "temperature": 54.93, "smoke": 282, "gas": 357, "humidity": 38, "location": "Salle_1", "scenario": "suspect"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:05", "temperature": 27.62, "smoke": 72, "gas": 89, "humidity": 56, "location": "Salle_1", "scenario": "normal"}
{"device_id": "fire_sensor_01", "timestamp": "2026-05-31T16:05:08", "temperature": 30.55, "smoke": 179, "gas": 84, "humidity": 56, "location": "Salle_1", "scenario": "normal"}
```

Figure 3 - Réception des messages JSON avec mosquitto_sub sur le topic capteurs.

5.3 Orchestration avec Node-RED

Node-RED assure le traitement intermédiaire. Le flow final reçoit les données MQTT, valide le payload, convertit les mesures en nombres, applique une logique de pré-alerte par seuils, prépare la commande pour le script IA, appelle `predict.py`, parse la réponse JSON, fusionne le résultat IA avec le message initial et publie une alerte si nécessaire.

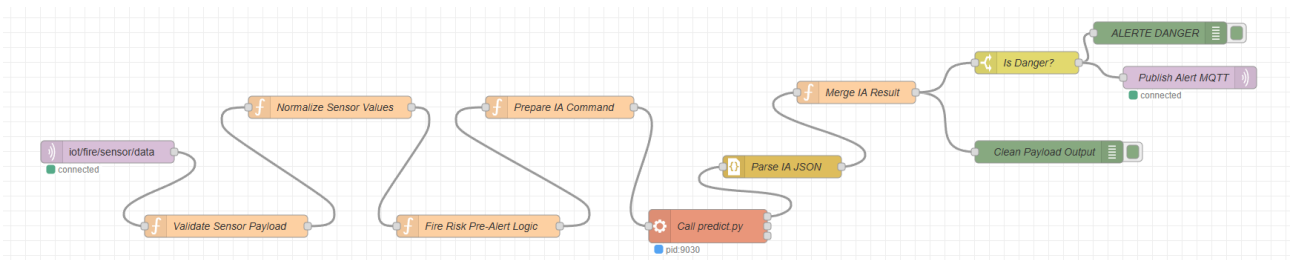


Figure 4 - Flow Node-RED final avec intégration IA et publication d'alerte.

5.4 Visualisation cloud avec ThingsBoard

La partie cloud repose sur ThingsBoard Cloud. Le tableau de bord présente les séries temporelles des mesures de capteurs et deux widgets de synthèse : l'alerte et le statut IA. Cette visualisation permet de vérifier rapidement l'évolution du système et de distinguer les trois états normal, suspect et danger.

5.5 Actionneur simulé et logique d'alerte

Dans ce prototype, l'actionneur est simulé sous forme d'alerte logicielle. Lorsqu'un message enrichi possède `alert: true`, Node-RED envoie ce message vers le debug `ALERTE DANGER` et vers un topic MQTT dédié : `iot/fire/alert`. Cette décision correspond à un actionneur virtuel. Dans une version matérielle, ce topic pourrait commander un buzzer, une LED rouge, un relais ou un système de notification.

Cette solution permet de valider la partie actionneur sans composant physique. Elle respecte néanmoins la logique IoT attendue : une donnée mesurée déclenche un traitement automatique, puis une action est produite lorsque la condition de danger est confirmée.

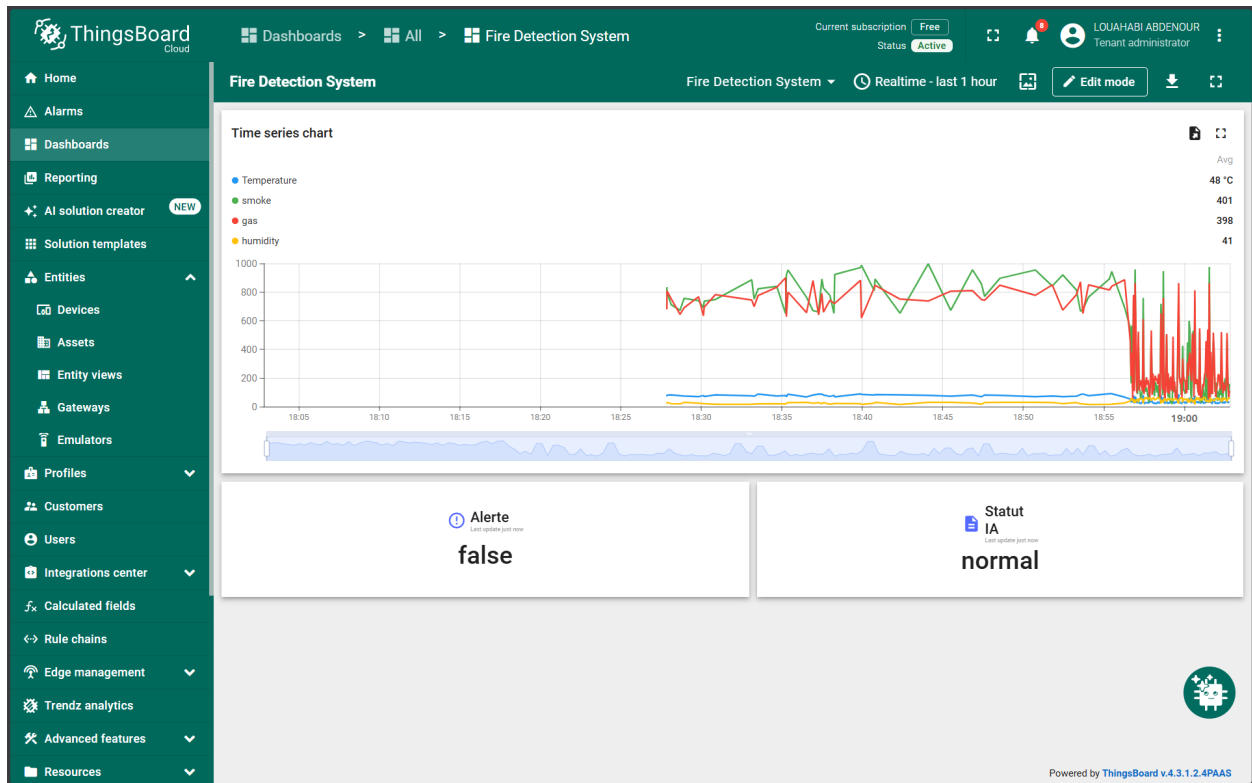


Figure 5 - Vue générale du tableau de bord ThingsBoard utilisé pour la visualisation.

6. Partie IA : données, modèle et intégration

6.1 Données utilisées

La partie IA utilise un dataset tabulaire simulé composé de 120 échantillons équilibrés. Chaque ligne contient quatre variables : température, fumée, gaz et humidité, ainsi qu'un label parmi `normal`, `suspect` et `danger`. Les données sont conçues pour refléter les plages de valeurs utilisées par le simulateur et permettre au modèle d'apprendre des frontières de classification simples.

6.1.1 Plages utilisées pour les scénarios

Classe	Température	Fumée	Gaz	Humidité
normal	20 à 35 °C	20 à 200	50 à 250	45 à 65 %
suspect	40 à 60 °C	250 à 600	300 à 550	30 à 45 %
danger	65 à 90 °C	650 à 1000	600 à 900	15 à 30 %

Tableau 2 - Plages de valeurs utilisées par le simulateur pour générer les trois scénarios.

Ces plages ne représentent pas une norme industrielle ; elles constituent un jeu de données pédagogique permettant de valider le pipeline complet. Le but n'est pas de remplacer un système certifié de sécurité incendie, mais de montrer comment un modèle IA peut être intégré dans une architecture IoT.

6.2 Modèle choisi

Le modèle retenu est un Random Forest Classifier. Ce choix est cohérent avec la nature des données : les variables sont numériques, peu nombreuses, et la sortie correspond à une classification multi-classes. Une forêt aléatoire combine plusieurs arbres de décision, ce qui améliore la stabilité des prédictions par rapport à un arbre unique. Le modèle est entraîné dans `train\_model.py`, sauvegardé sous forme de fichier `model\_fire\_detection.joblib`, puis chargé par `predict.py` lors de l'inférence.

6.3 Intégration de l'IA dans Node-RED

L'intégration repose sur un node `exec` qui appelle le script Python `ai/predict.py`. Le node `Prepare IA Command` extrait les quatre valeurs nécessaires et les transmet sous forme d'arguments. Le script

retourne une sortie JSON contenant `ai\_status`, `risk\_score` et `alert`. Le node `Merge IA Result` fusionne ensuite cette sortie avec le message capteur original.

Cette approche montre une manière simple d'intégrer une brique IA externe dans Node-RED sans créer d'API dédiée. Elle est suffisante pour une démonstration académique et peut évoluer vers une API Flask/FastAPI ou un service cloud dans une version future.

7. Méthodologie de validation expérimentale

La validation a été organisée en plusieurs niveaux afin de distinguer les erreurs possibles. Chaque composant a d'abord été testé séparément, puis la chaîne complète a été observée en streaming. Cette méthodologie évite de confondre un problème de génération, de transport MQTT, de flow Node-RED ou de prédiction IA.

Étape	Commande ou observation	Résultat attendu	Preuve
Simulation	python simulator/simulate_fire_sensors.py	Messages JSON générés et publiés	Figure 2
MQTT	mosquitto_sub -h localhost -t iot/fire/sensor/data	Réception de plusieurs scénarios	Figure 3
IA seule	python ai/predict.py ...	Classe normal/suspect/danger	Figure 6
Node-RED	Debug panel	Message enrichi avec ai_status et alert	Figure 8
Cloud	Dashboard ThingsBoard	Visualisation des états IA	Figures 9 à 11

La présence simultanée du champ `scenario` et du champ `ai\_status` ne signifie pas que la donnée source contient déjà la prédiction. Le champ `scenario` sert à générer des valeurs réalistes et vérifiables ; le champ `ai\_status` est ajouté après l'appel du modèle IA. Cette distinction est essentielle pour montrer que le système réalise bien une classification en flux.

8. Résultats et démonstration

7.1 Prédiction manuelle du modèle IA

Avant de valider le streaming complet, le script `predict.py` a été testé manuellement avec trois entrées représentatives. Les sorties confirment que le modèle reconnaît correctement les états normal, suspect et danger. L'alerte reste fausse pour normal et suspect, et devient vraie uniquement en cas de danger.

```
(youssef) youssef@Youssef:/mnt/e/projects/iot-fire-detection-ai/ai$ cd /mnt/e/projects/iot-fire-detection-ai
python ai/predict.py 24 80 120 55
python ai/predict.py 48 340 300 40
python ai/predict.py 78 850 720 22
{"ai_status": "normal", "risk_score": 1.0, "alert": false}
{"ai_status": "suspect", "risk_score": 1.0, "alert": false}
{"ai_status": "danger", "risk_score": 1.0, "alert": true}
(youssef) youssef@Youssef:/mnt/e/projects/iot-fire-detection-ai$
```

Figure 6 - Tests manuels du script predict.py sur les trois scénarios.

7.2 Évaluation du modèle

L'évaluation sauvegardée indique que le dataset contient 120 échantillons, dont 90 pour l'entraînement et 30 pour le test. Le modèle Random Forest obtient une accuracy de 100 % sur le jeu de test, avec une cross-validation 5-fold également à 100 %. La matrice de confusion montre 10 prédictions correctes pour chaque classe et aucune confusion entre les classes. Ces résultats sont cohérents avec le caractère contrôlé et bien séparé du dataset simulé.

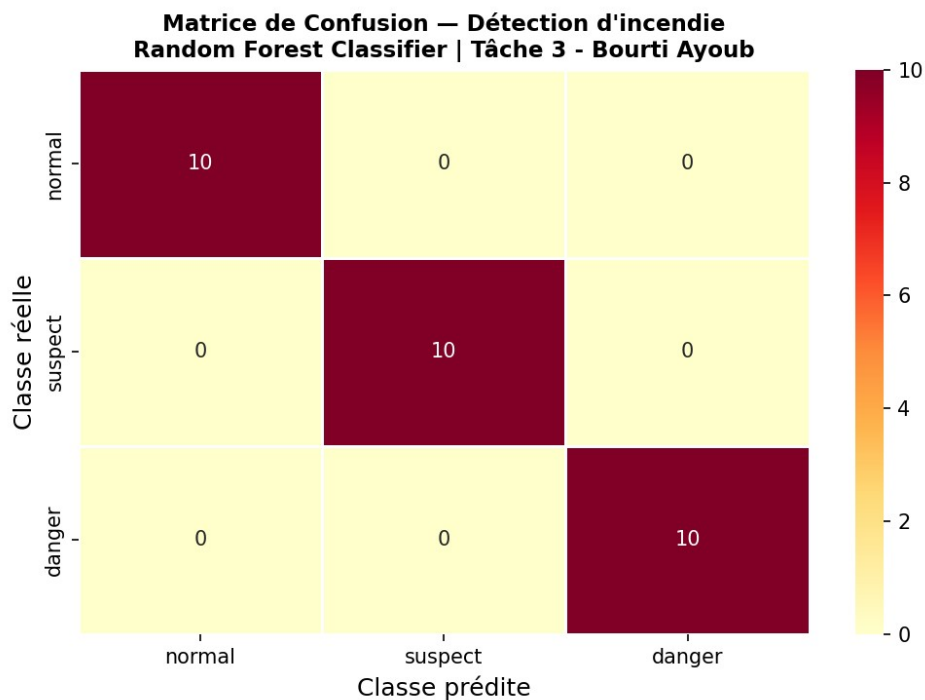


Figure 7 - Matrice de confusion du modèle IA sur les trois classes.

7.3 Synthèse quantitative

Indicateur	Valeur	Interprétation	Source
Échantillons	120	Dataset simulé équilibré	evaluation_results.txt
Train / Test	90 / 30	Séparation 75 % / 25 %	train_model.py
Classes	normal, suspect, danger	Classification multi-classes	dataset_fire_detection.csv
Accuracy test	100 %	Aucune erreur sur le test contrôlé	evaluation_results.txt
Feature principale	smoke	Importance la plus élevée dans le modèle	evaluation_results.txt

7.4 Prédiction en streaming dans Node-RED

La validation la plus importante consiste à observer les prédictions en streaming. Dans ce cas, les valeurs proviennent du simulateur, transitent par MQTT, sont traitées par Node-RED, puis classées par l'IA. Les captures suivantes montrent que les champs `pre\_alert`, `ai\_status`, `risk\_score` et `alert` sont ajoutés après traitement. Le scénario danger déclenche bien `alert: true`.

Normal	Suspect	Danger
<pre> 5/31/2026, 4:14:40 PM node: Clean Payload Output iot/fire/sensor/data : msg.payload : Object ▼ object device_id: "fire_sensor_01" timestamp: "2026-05-31T16:14:19" temperature: 21.61 smoke: 114 gas: 185 humidity: 45 location: "Salle_1" scenario: "normal" pre_alert: "normal" ai_status: "normal" risk_score: 0.99 alert: false </pre>	<pre> 5/31/2026, 4:14:44 PM node: Clean Payload Output iot/fire/sensor/data : msg.payload : Object ▼ object device_id: "fire_sensor_01" timestamp: "2026-05-31T16:14:22" temperature: 41.48 smoke: 511 gas: 474 humidity: 45 location: "Salle_1" scenario: "suspect" pre_alert: "suspect" ai_status: "suspect" risk_score: 1 alert: false </pre>	<pre> 5/31/2026, 4:14:37 PM node: ALERTE DANGER iot/fire/sensor/data : msg.payload : Object ▼ object device_id: "fire_sensor_01" timestamp: "2026-05-31T16:14:16" temperature: 79.35 smoke: 851 gas: 836 humidity: 23 location: "Salle_1" scenario: "danger" pre_alert: "danger" ai_status: "danger" risk_score: 1 alert: true </pre>

Figure 8 - Résultats Node-RED pour les scénarios normal, suspect et danger.

7.5 Visualisation ThingsBoard

Les captures ThingsBoard montrent la visualisation cloud des mesures et de l'état final. Le tableau de bord affiche les courbes de température, fumée, gaz et humidité, ainsi qu'un widget `Alerte` et un widget `Statut IA`. Les trois états sont visibles : normal sans alerte, suspect sans alerte, et danger avec alerte.

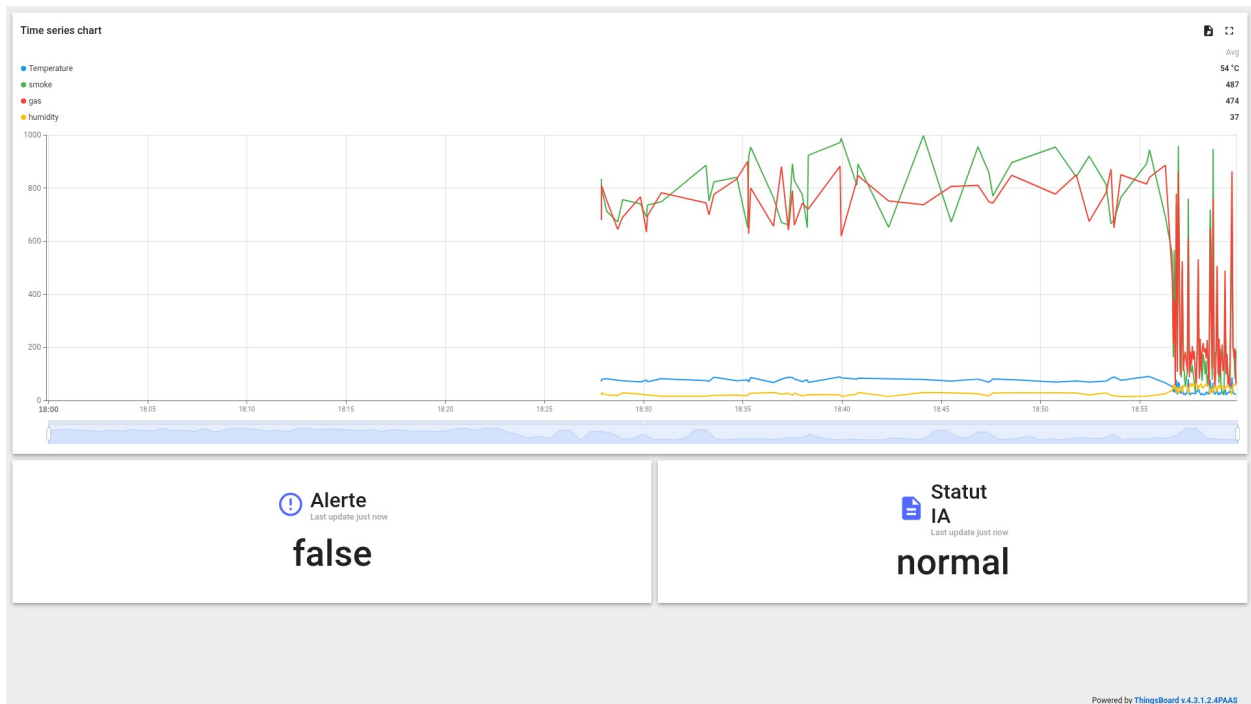


Figure 9 - Tableau de bord ThingsBoard : scénario normal, alerte false.

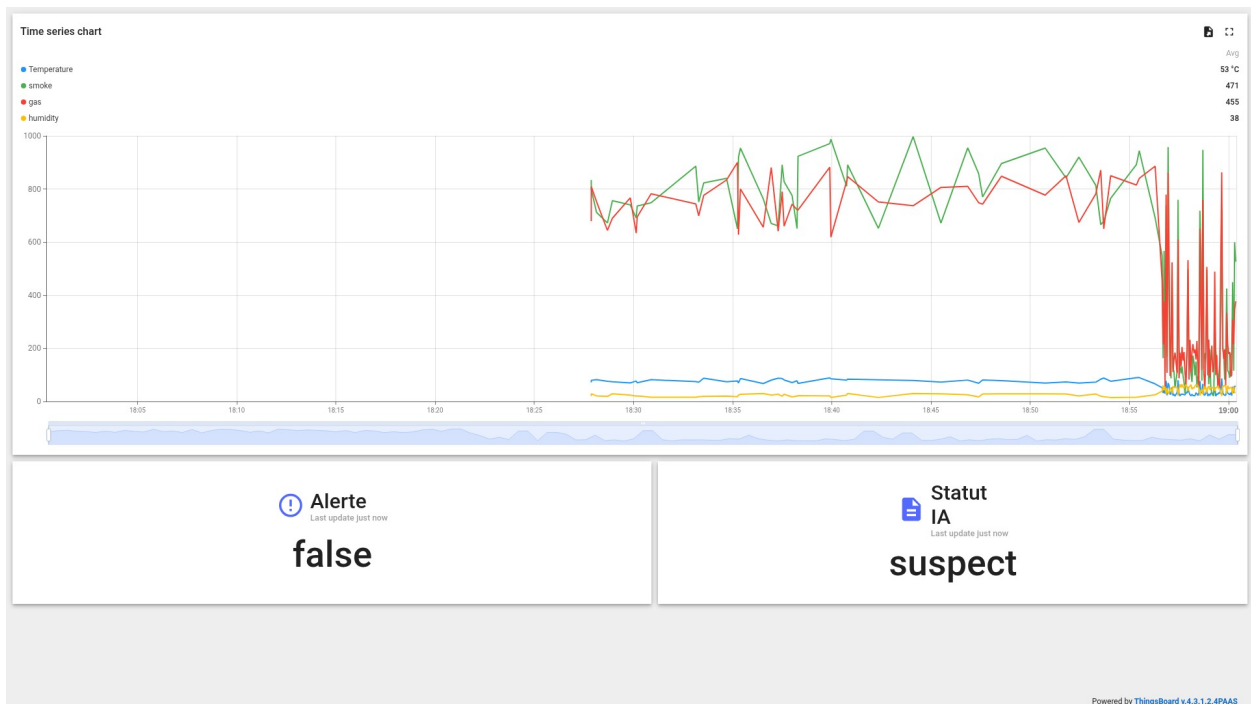


Figure 10 - Tableau de bord ThingsBoard : scénario suspect, alerte false.



Figure 11 - Tableau de bord ThingsBoard : scénario danger, alerte true.

9. Discussion, limites et perspectives

Les résultats obtenus valident le fonctionnement global du prototype. Le système couvre les principales couches demandées : capteurs simulés, communication MQTT, orchestration Node-RED, décision automatique, composante IA et visualisation cloud. La démonstration montre également la différence entre le label de scénario généré par le simulateur et la prédiction IA ajoutée par `predict.py`. Le champ `scenario` sert à produire des données réalistes et à vérifier les tests, tandis que `ai\_status` correspond à la décision du modèle.

La principale limite du projet vient du fait que les données sont simulées. Les classes sont volontairement séparées par des plages de valeurs distinctes, ce qui explique les performances très élevées. Dans un environnement réel, les capteurs peuvent être bruités, mal calibrés ou soumis à des cas intermédiaires. Il faudrait donc collecter davantage de données réelles, introduire du bruit et tester la robustesse du modèle.

Une autre limite concerne l'intégration IA via un appel de script Python dans Node-RED. Cette solution est simple et efficace pour une démonstration, mais une architecture plus robuste pourrait utiliser une API REST locale ou un microservice conteneurisé. Cela permettrait une meilleure gestion des erreurs, des logs et de la montée en charge.

Comme perspectives, le système pourrait être étendu avec de vrais capteurs sur Arduino ou Raspberry Pi, un actionneur réel tel qu'un buzzer ou une LED, un stockage historique, une gestion d'utilisateurs, des notifications par e-mail ou Telegram, ainsi qu'une comparaison entre plusieurs modèles IA. Une version avancée pourrait aussi intégrer la détection d'anomalies non supervisée afin d'identifier des comportements inconnus.

10. Structure technique du dépôt

Le dépôt final est organisé par responsabilités techniques. Le dossier `simulator` contient la couche edge, `flow` et `integration\_ia\_node\_red` contiennent les flows Node-RED, `ai` contient le dataset, l'entraînement, le modèle et la prédiction, `cloud` contient les preuves de visualisation, et `screenshots` regroupe les captures utilisées pour la validation.

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ find . -maxdepth 2 -type f | sort
./gitignore
./PROJECT_STATUS.md
./README.md
./ai/confusion_matrix.png
./ai/dataset_fire_detection.csv
./ai/evaluation_results.txt
./ai/model_fire_detection.joblib
./ai/predict.py
./ai/train_model.py
./article_screenshots/01_simulator_mqtt_publish.png
./article_screenshots/02_mqtt_subscriber_received_json.png
./article_screenshots/03_node_red_final_flow.png
./article_screenshots/04_node_red_debug_normal.png
./article_screenshots/05_node_red_debug_suspect.png
./article_screenshots/06_node_red_debug_danger_alert.png
./article_screenshots/07_ai_prediction_tests.png
./article_screenshots/08_confusion_matrix.png
./article_screenshots/09_evaluation_results.txt
./article_screenshots/dashboard_danger.png
./article_screenshots/dashboard_normal.png
./article_screenshots/dashboard_suspect.png
./article_screenshots/thingsboard_dashboard.png
./cloud/README_demo.md
./cloud/preuve_test_bout_en_bout.md
./config.example.json
./docs/README_node_red_base(membre2).md
./docs/README_task3.md
./docs/checklist_soutenance.md
./docs/mqtt_format_membre1.md
./docs/validation_membre1.md
./flow/flow_node_red_base.json
./integration_ia_node_red/README_integration_ia.md
./integration_ia_node_red/flow_node_red_ai_alert.json
./iot-fire-detection-ai.pdf
./logs/sample_output.txt
./requirements.txt
./screenshots/capture_node_red_reception.png
./screenshots/danger_alert.png
./screenshots/dashboard_danger.png
./screenshots/dashboard_normal.png
./screenshots/dashboard_suspect.png
./screenshots/normal_alert.png
./screenshots/suspect_alert.png
./screenshots/thingsboard_dashboard.png
./simulator/config.json
./simulator/simulate_fire_sensors.py
./tests/mqtt_subscriber_test.py
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$
```

Figure 12 - Structure du dépôt du projet.

11. Conclusion

Ce projet a permis de concevoir un prototype complet de détection intelligente d'incendie basé sur l'IoT et l'intelligence artificielle. La chaîne réalisée couvre la génération de données capteurs, la publication MQTT, l'orchestration Node-RED, la classification IA, l'alerte et la visualisation cloud. Les tests montrent que le système détecte correctement les états normal, suspect et danger, aussi bien en prédiction manuelle qu'en flux temps réel dans Node-RED.

Le projet répond aux objectifs pédagogiques du module en intégrant les concepts IoT étudiés en cours et en ajoutant une brique IA autonome. Même si le prototype reste simulé, il constitue une base claire pour une extension vers un système matériel réel et vers une architecture plus robuste de supervision incendie.

12. Références

- [1] Node-RED, Documentation officielle, <https://nodered.org/docs/>
- [2] MQTT.org, MQTT - The Standard for IoT Messaging, <https://mqtt.org/>
- [3] OASIS, MQTT Version 5.0 Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [4] ThingsBoard, Documentation officielle - Telemetry and Dashboards, <https://thingsboard.io/docs/>
- [5] Scikit-learn, RandomForestClassifier documentation, <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [6] Cahier des charges du module : Projet de fin de module & recherche, Master IASD, 2025/2026.
- [7] Dépôt du projet : iot-fire-detection-ai, fichiers techniques et captures de validation.